

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

João Victor Serrano

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	4
4.1	Atividade 1 — Agenda telefônica em Flask	4
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	6
4.4	Atividade 4 — Controle de acesso por papel — RBAC	8
4.5	Atividade 5 — Logs e auditoria	10
4.6	Atividade 6 — Upload e perfil de usuário	12
5	Atividades extras (opcional)	14
5.1	Atividade E1 — Documentação Swagger/OpenAPI	14
5.2	Atividade E2 — CI/CD com GitHub Actions	16
5.3	Atividade E3 — Observabilidade — health checks e métricas	18
6	Considerações finais	20
7	Declaração de autoria	20

1 Introdução

A disciplina Introdução à Computação em Nuvem trata de como uma aplicação sai da máquina do desenvolvedor e passa a rodar de forma confiável em um ambiente de nuvem: empacotada em contêineres, dividida em serviços que conversam por uma rede, apoiada em serviços de infraestrutura (banco de dados, cache, armazenamento de objetos) e acompanhada por automação e monitoramento. Em vez de exercícios isolados, o bimestre foi conduzido como a evolução de um único projeto, de modo que cada atividade acrescenta uma peça ao que a anterior deixou pronto.

O fio condutor foi um catálogo de filmes de Tom Hanks. A atividade inicial era um nivelamento em Flask (uma agenda telefônica). A partir da segunda, o catálogo consome a API pública do TMDb, grava favoritos e comentários em um banco MariaDB e separa os dados por usuário. Nas semanas seguintes o sistema foi sendo decomposto e ampliado: o login saiu do backend e virou um microsserviço de autenticação na rede interna do Docker; o campo de papel do usuário passou a decidir permissões reais no servidor (RBAC); um serviço de logs com Redis passou a registrar login, ações sensíveis e acessos negados; e a página de perfil ganhou foto, guardada em um serviço de armazenamento de objetos (MinIO) com apenas a referência no banco. Três atividades extras, sem prazo, acrescentaram documentação Swagger/OpenAPI, um pipeline de CI/CD e observabilidade.

Este relatório reúne, em ordem, o que foi entregue em cada atividade. Depois da metodologia há um quadro-resumo com data planejada, data realizada e situação; em seguida, uma ficha por atividade com a descrição, o link direto do commit, o que foi construído, os prints de evidência e as dificuldades encontradas. A Atividade 1 não foi entregue e consta com o motivo, e a Atividade 5 foi entregue com atraso, o que também é explicado na própria ficha.

2 Metodologia

Todas as atividades de 2 a 6 e as extras foram desenvolvidas no mesmo repositório público do GitHub (JoaoVSerrano/tomhanks). O sistema é composto por um gateway em Flask que também serve o frontend Angular já compilado, o auth-service e o log-service (ambos em Flask), um Redis, um MinIO, Prometheus e Grafana, além de um MariaDB externo fornecido para a turma. Tudo é orquestrado com Docker Compose e publicado em uma stack no Portainer. As configurações sensíveis (chaves, senhas e dados de conexão) são lidas de variáveis de ambiente, e há uma suíte de testes automatizados com pytest. O trabalho foi feito de forma incremental, com commits ao longo do desenvolvimento, e a análise deste relatório se baseia nesse histórico.

As datas e horas de entrega saíram do histórico de commits, extraídas com `git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad %s" <hash>`, e a conferência visual é registrada nos prints da página de cada commit no GitHub. Todos os commits do repositório carregam o fuso -03:00 (horário de Brasília), de modo que nenhuma conversão foi necessária. Nenhuma data ou hora foi estimada: quando não há commit que comprove uma atividade, ela consta como não entregue.

O commit escolhido para representar cada entrega é aquele em que a funcionalidade pedida passou a funcionar de forma completa. As correções feitas depois, quando existem, são citadas na ficha da atividade, com hash e horário, em vez de substituírem a data da entrega. A situação é definida comparando o horário desse commit com a data planejada, considerando o prazo até as 23:59 do dia: “entregue” quando está dentro do prazo, “entregue com atraso” quando passou, e “não entregue” quando não há evidência. Nas atividades extras, que não têm prazo, vale o mesmo critério de completude: nas extras E1 e E2 a versão

inicial já existia em 30/09, mas o requisito só ficou completo em commits posteriores, que são os usados como data de entrega, com a versão inicial citada na ficha.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	—	não entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 23:59	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	28/08/2026 17:04	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	04/09/2026 15:40	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	30/09/2026 19:52	entregue com atraso
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	30/09/2026 20:04	entregue
E1	Documentação Swagger/OpenAPI	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.	sem prazo	05/10/2026 21:26	entregue
E2	CI/CD com GitHub Actions	Pipeline que testa e faz deploy a cada push.	sem prazo	05/10/2026 21:27	entregue
E3	Observabilidade — health checks e métricas	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.	sem prazo	30/09/2026 20:43	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	—
Situação	não entregue
Evidência	Sem evidência: não há repositório, commit nem print desta atividade
Link	—

O que foi feito. Nada foi entregue nesta atividade. Não existe repositório, commit ou captura de tela da agenda telefônica, e por isso nenhuma data ou link é informado.

O que faltou e por quê. A agenda em Flask com Jinja e persistência em JSON não foi desenvolvida. O motivo foi a agenda do bimestre e a decisão de dedicar a carga horária às atividades seguintes, que valiam mais para o projeto principal.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 23:59
Situação	entregue
Evidência	GitHub — commit ed42efb + README + print do catálogo
Link	https://github.com/JoaoVSerrano/tomhanks/commit/ed42efb

O que foi feito. O backend em Flask (servido com Gunicorn) consulta a API do TMDb para montar o catálogo (/api/catalog), com a chave guardada em variável de ambiente. Sobre o catálogo o usuário pode favoritar filmes (/api/favorites) e comentar (/api/comments), e esses dados ficam em um MariaDB, nas tabelas favoritos e comentarios. Ambas têm a coluna usuario_id, e as consultas sempre filtram pelo usuário logado, o que garante a segregação: cada pessoa só vê e só apaga o que é seu. Nesta etapa o login ainda fazia parte do mesmo backend, com sessão do Flask; ele só foi extraído na Atividade 3.

Uma decisão de arquitetura foi trocar o SQL escrito à mão da primeira versão (095a624, 20:51) por SQLAlchemy com migrações Alembic (d3975ed, 22:43), para que o esquema do banco fosse versionado junto com o código. O sistema roda em contêiner com Dockerfile e docker-compose.yml, com o frontend Angular já compilado servido pelo próprio Flask, e o banco MariaDB é externo, configurado por variáveis de ambiente.

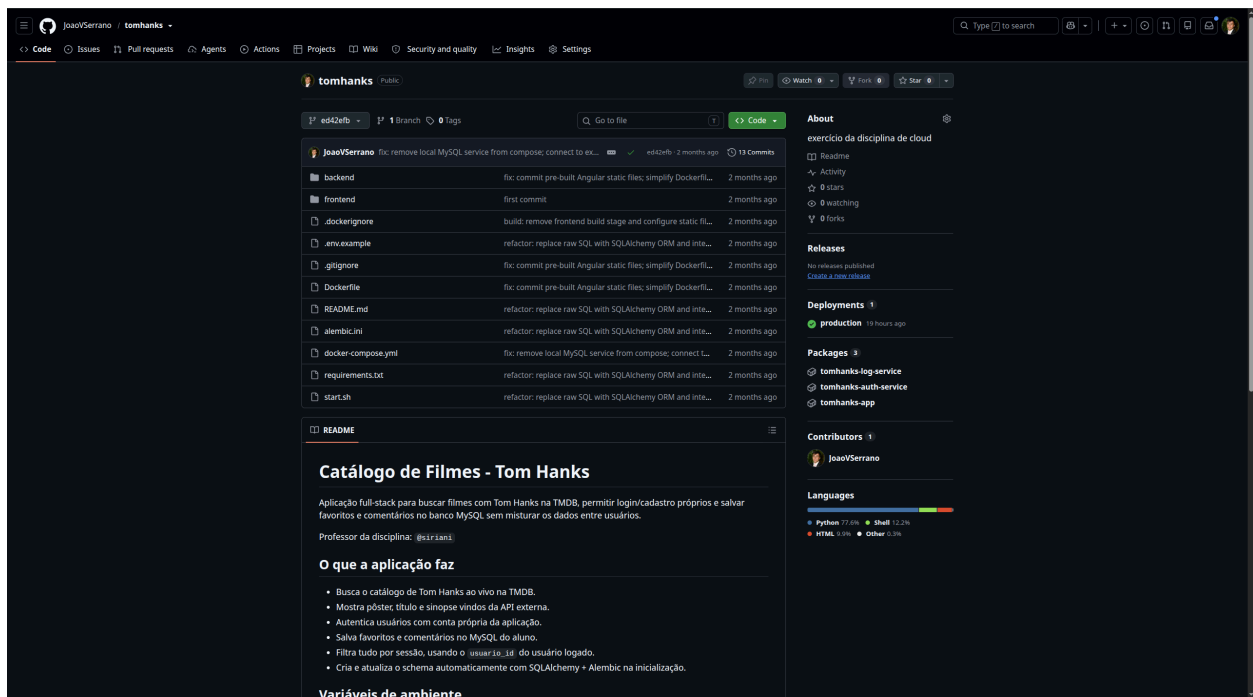


Figura 1: Atividade 2 — evidência da entrega: página do commit ed42efb no GitHub, com hash, data/hora e nome do repositório visíveis

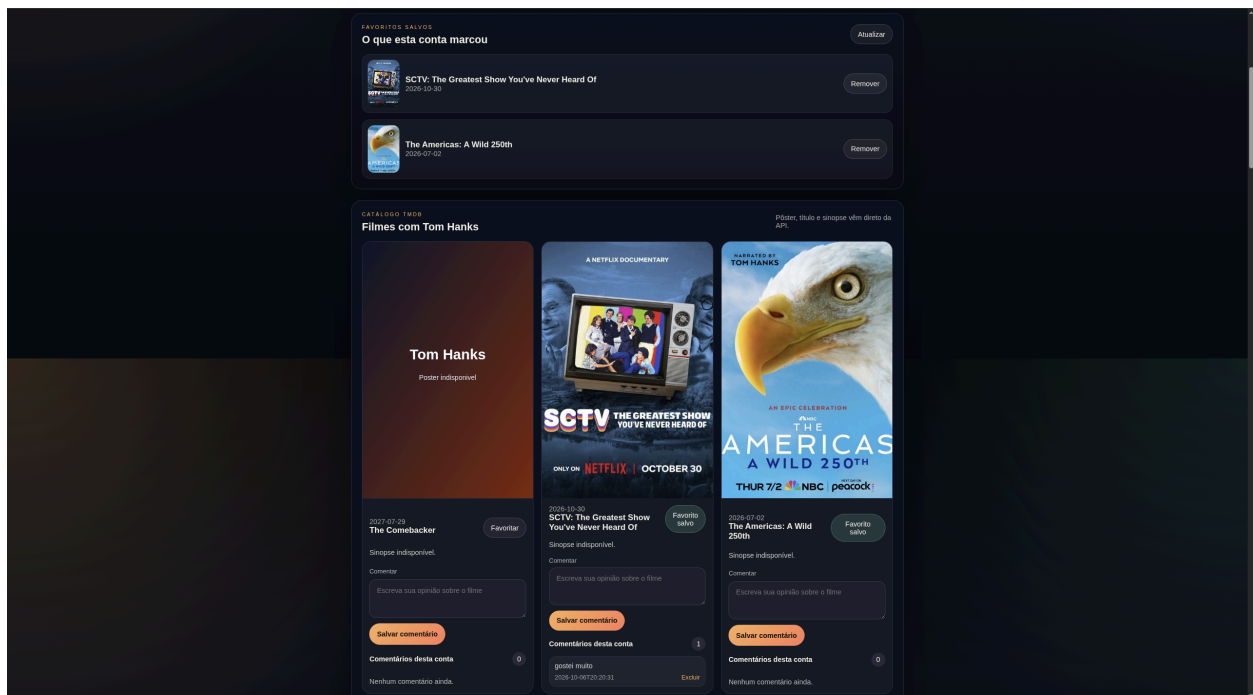


Figura 2: Atividade 2 — resultado: catálogo de filmes carregado a partir do TMDb, com favoritos e comentários do usuário logado

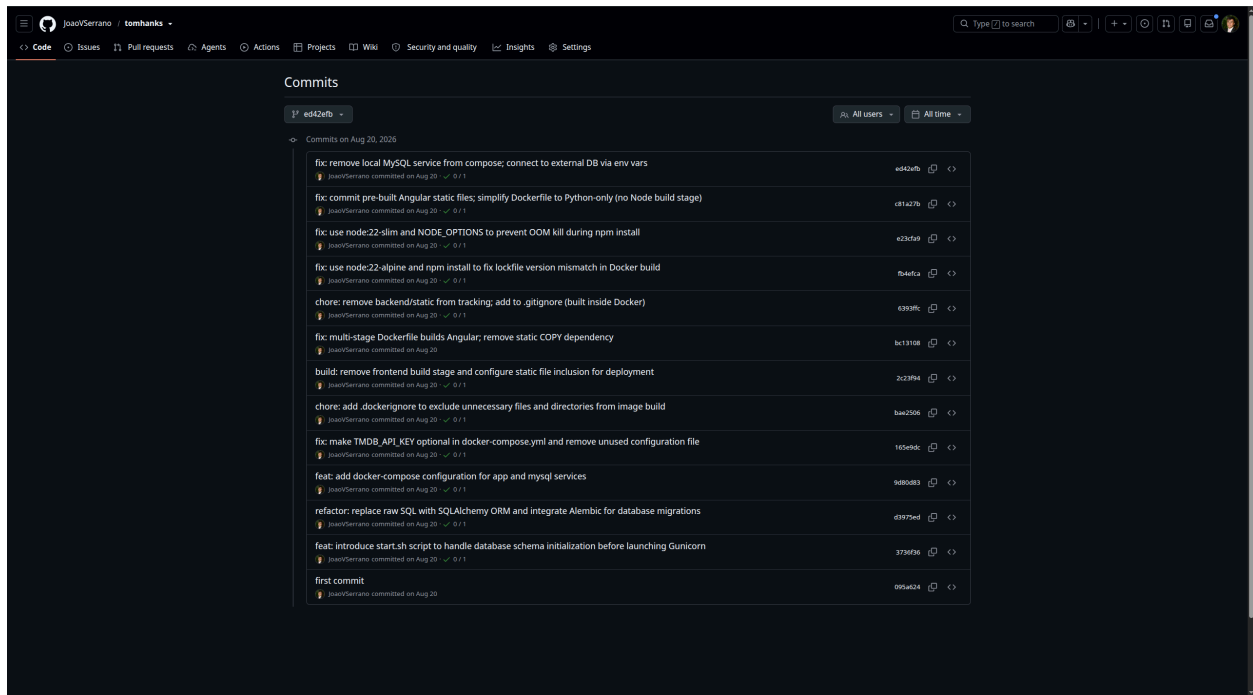


Figura 3: Atividade 2 — detalhe dos commits realizados até a entrega ed42efb

Dificuldades e como foram resolvidas. A maior dificuldade foi empacotar o frontend. Na noite da entrega, a compilação do Angular dentro do Docker falhou repetidas vezes: primeiro por incompatibilidade de versão do arquivo de lock, depois por falta de memória durante o `npm install` (commits fb4efca e e23cfa9). A saída foi versionar os arquivos já compilados e deixar o Dockerfile só com Python (c81a27b, 23:57). Em seguida, o commit ed42efb (23:59) removeu o MySQL local do compose e passou a conectar no banco externo por variáveis de ambiente. A entrega aconteceu, portanto, no último minuto do prazo.

Correções posteriores. Em 02/10 o commit 0250c3d definiu valores padrão de conexão com o banco dentro do código para facilitar a implantação no Portainer. Em 05/10, o commit ca41bbc (21:23) removeu esses valores e tornou as variáveis do banco obrigatórias, fornecidas apenas pelo ambiente, e acrescentou testes que verificam essa exigência.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	28/08/2026 17:04
Situação	entregue
Evidência	GitHub — commit 2e93fab + docker-compose.yml + print do login funcionando
Link	https://github.com/JoaoV Serrano/tomhanks/commit/2e93fab

O que foi feito. A lógica de usuários saiu do backend e foi para um serviço próprio, o auth-service (auth_service/), executado em outro contêiner da rede tomhanks-net. Ele expõe cadastro (/register), login (/login), logout, usuário atual (/me) e o fluxo de recuperação de senha: /forgot-password gera um token com validade de 30 minutos e envia o link por e-mail via SMTP, /reset-password/check confere se o link ainda vale e /reset-password grava a nova senha. As senhas são guardadas como hash, e a sessão

usa cookie com a opção HttpOnly. O serviço tem as suas próprias tabelas (usuarios e reset_tokens), com migrações Alembic separadas.

O gateway deixou de conhecer usuários: as rotas /api/auth/* apenas repassam a chamada ao auth-service, e a consulta de dados de um usuário entre serviços usa uma rota interna protegida por um token compartilhado (X-Internal-Token). O modelo de usuário foi removido do backend, que passou a guardar só o identificador (usuario_id) nas suas tabelas.

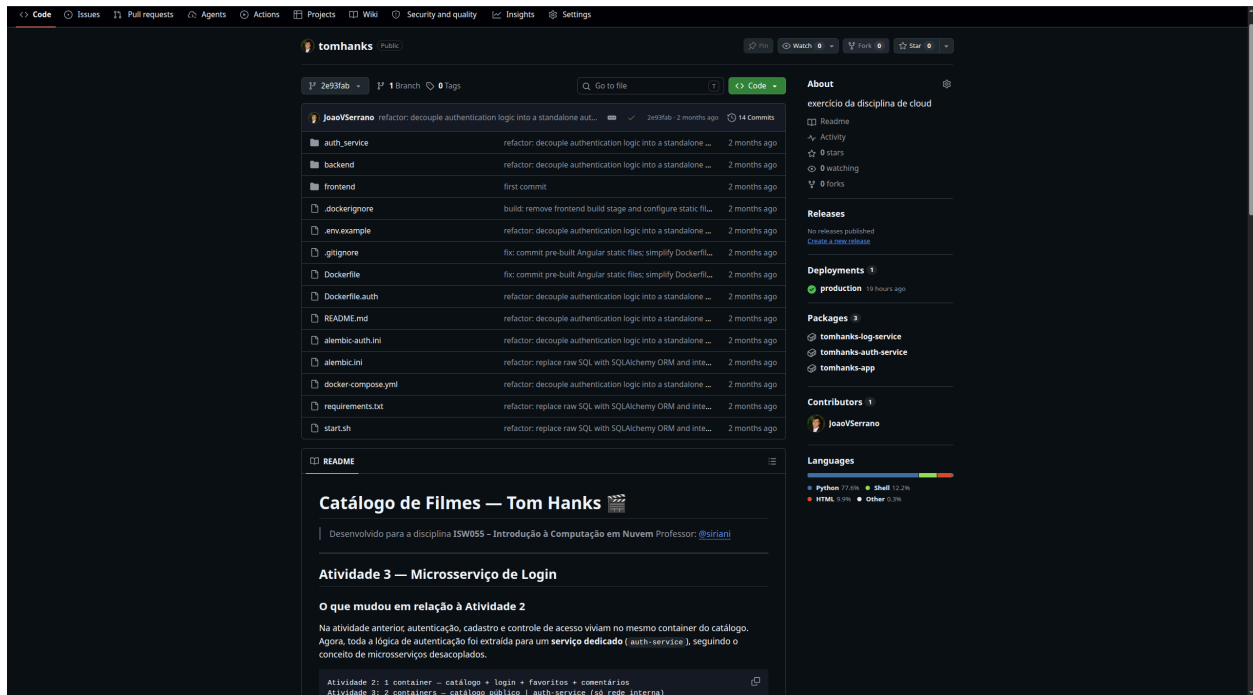


Figura 4: Atividade 3 — evidência da entrega: página do commit 2e93fab no GitHub, com hash, data/hora e nome do repositório visíveis

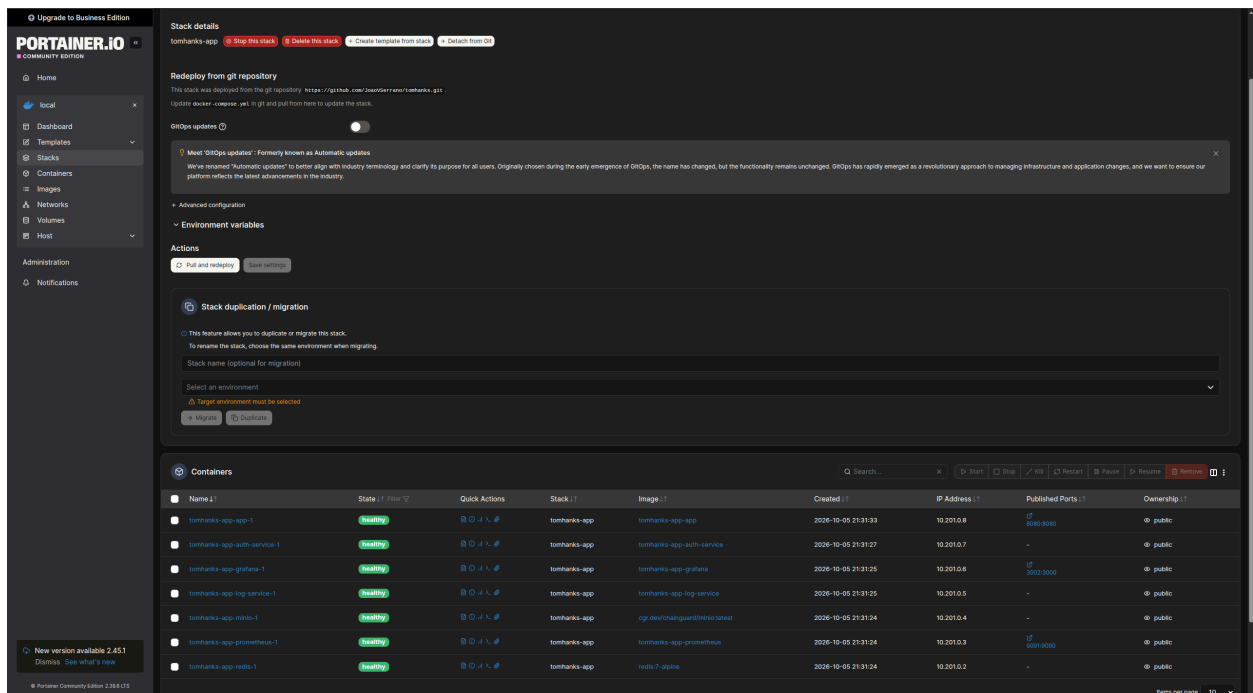


Figura 5: Atividade 3 — resultado: tela de login funcionando, com o auth-service aparecendo em contêiner separado (por exemplo docker compose ps ou a stack no Portainer)

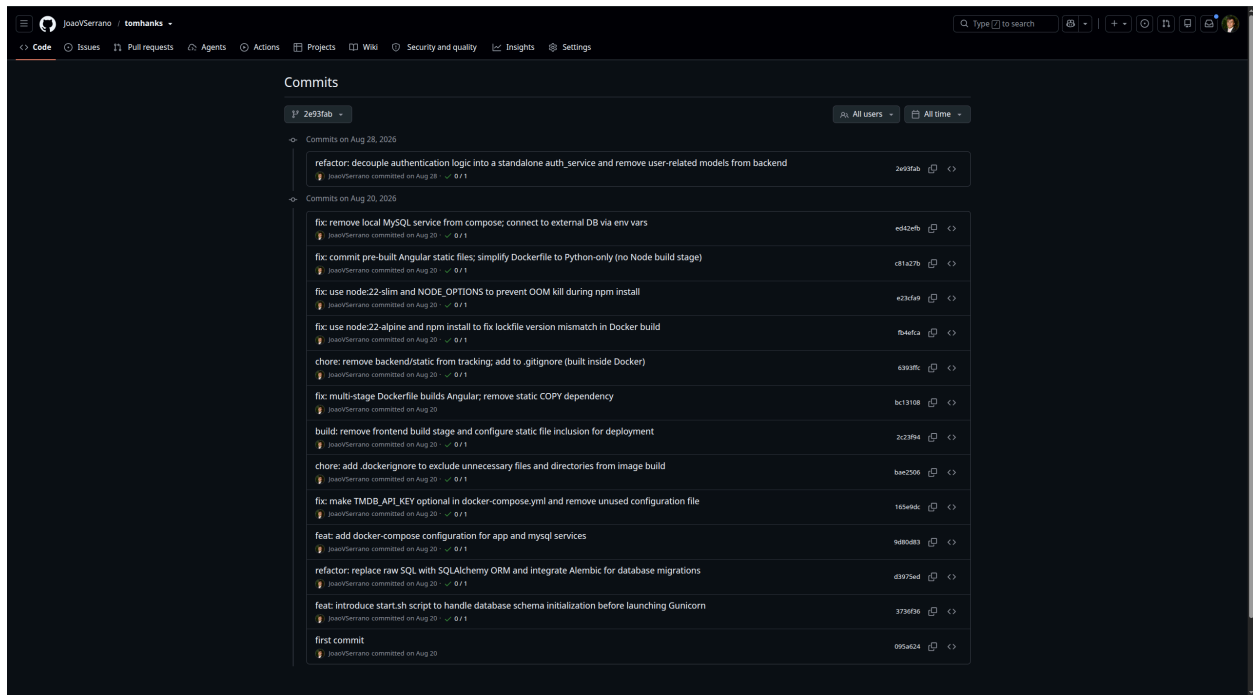


Figura 6: Atividade 3 — detalhe dos commits realizados até a entrega 2e93fab:

Dificuldades e como foram resolvidas. O histórico mostra ajustes na rede do Docker Compose logo depois da entrega (ebff172, 28/08 17:44) e novos ajustes de rede em 04/09, descritos na Atividade 4. Em 02/10, o commit c4638bb (18:08) corrigiu um problema de estabilidade do auth-service: limitou os workers do Gunicorn, tornou a inicialização do banco segura contra chamadas simultâneas e fez o healthcheck deixar de depender do MinIO.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	04/09/2026 15:40
Situação	entregue
Evidência	GitHub — commit bb1fa6b + print do 403 e da ação de admin
Link	https://github.com/JoaoVSerrano/tomhanks/commit/bb1fa6b

O que foi feito. A tabela usuarios já tinha o campo role, com os valores usuario (padrão) e admin, e ele passou a ser a base das permissões, verificadas sempre no servidor e nunca apenas na interface. No gateway foi criada a função require_role (401 se não há sessão, 403 se o papel não basta). No auth-service, as rotas administrativas /admin/users e /admin/users/<id>/role só respondem para quem é administrador, e a troca de papel só aceita os valores usuario e admin. O primeiro administrador é criado na inicialização a partir de variáveis de ambiente.

A regra mais visível está nos comentários: um usuário comum só apaga os próprios, e se tentar apagar o de outra pessoa recebe 403; o administrador pode moderar e apagar qualquer comentário. As rotas /api/admin/* do gateway apenas repassam a chamada ao auth-service, que é quem decide.

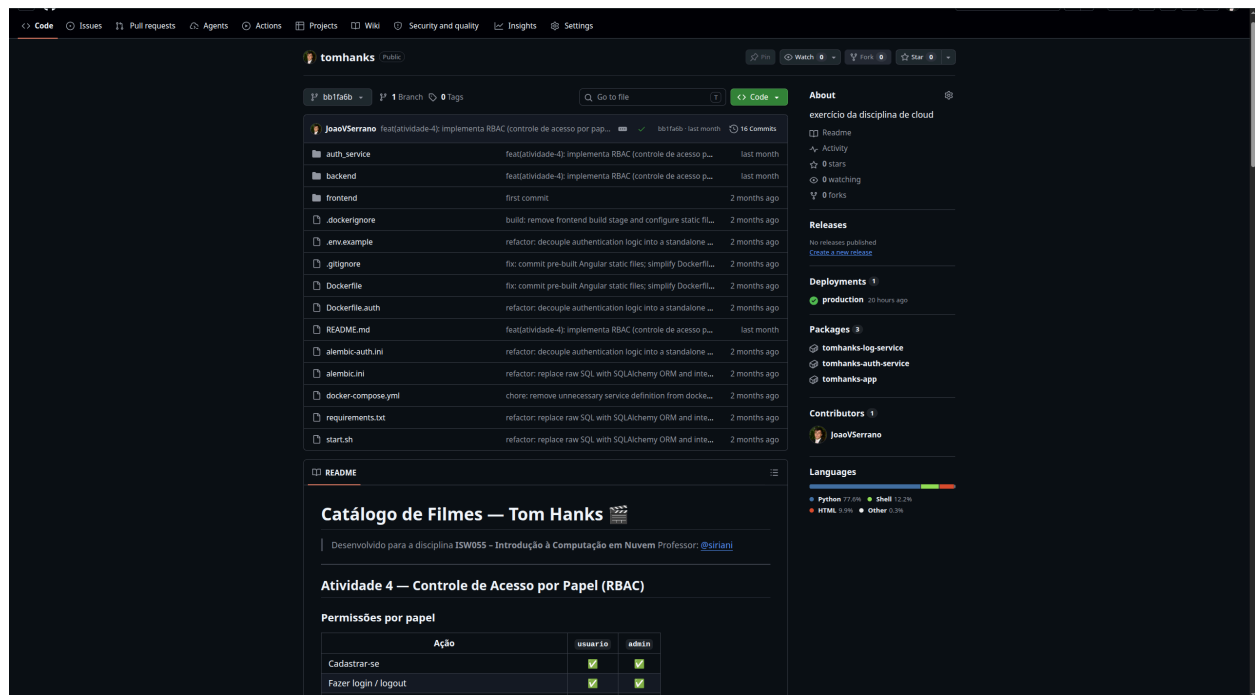


Figura 7: Atividade 4 — evidência da entrega: página do commit bb1fa6b no GitHub, com hash, data/hora e nome do repositório visíveis



Figura 8: Atividade 4 — resultado: administrador com os privilégios garantidos por RBAC na comanda da atividade

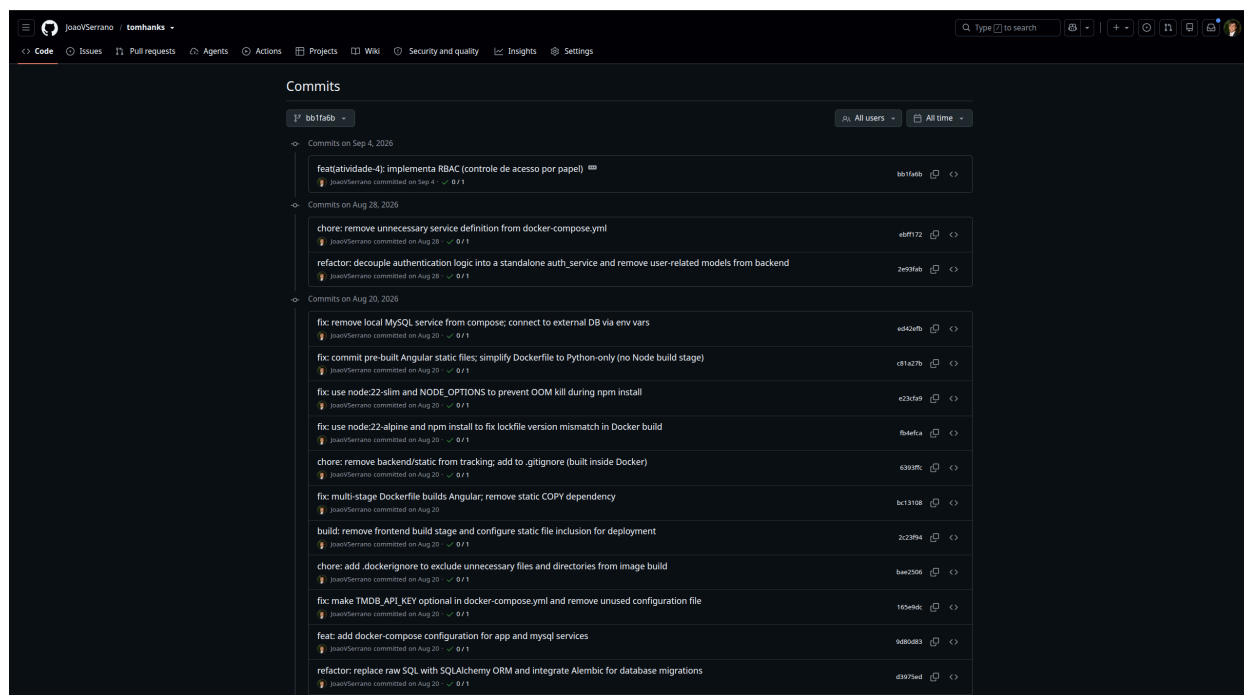


Figura 9: Atividade 4 — detalhe dos commits até a entrega bb1fa6b

Dificuldades e como foram resolvidas. Na mesma tarde da entrega houve três correções de infraestrutura ligadas ao Portainer: a remoção de uma sub-rede fixa da rede tomhanks-net (6f0393d, 16:04), a definição de uma sub-rede 10.200.200.0/28 fora do pool padrão do Docker (f3480b9, 16:06) e a retirada dos nomes fixos de contêiner (container_name) para evitar conflito entre stacks (27a3b97, 16:18). O padrão foi o mesmo nas três: algo que funcionava localmente colidia com o ambiente compartilhado do Portainer.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	30/09/2026 19:52
Situação	entregue com atraso
Evidência	GitHub — commit 43be2ea + print da consulta de logs pelo admin
Link	https://github.com/JoaoVSerrano/tomhanks/commit/43be2ea

Motivo do atraso. A Atividade 5 foi entregue com atraso em relação à data planejada (25/09/2026). O atraso decorreu de período de ausência do aluno, motivado por feriados, cancelamentos de aula devido à formatura de alunos veteranos e por afastamento por motivo de saúde, entre 11/09/2026 e 28/09/2026. O histórico do repositório não registra commits entre 04/09/2026 16:18 e 30/09/2026 19:52.

O que foi feito. Foi criado o log-service, um microserviço Flask na rede interna (porta 4000, sem exposição ao host) que grava os eventos de auditoria em um Redis Stream (audit:logs), com limite de tamanho para a lista não crescer sem fim. Ele oferece POST /log, para gravar, e GET /logs, para consultar os últimos eventos (50 por padrão, no máximo 500), e as duas rotas só aceitam chamadas internas com o X-Internal-Token. O gateway registra, por meio de uma função auxiliar, os eventos de login, login que falhou, logout, favoritar e desfavoritar, comentar, apagar comentário (próprio ou por moderação) e as tentativas negadas com 403.

A consulta pelo usuário é feita em `/api/admin/logs`, que exige o papel de administrador. Um usuário comum que tenta acessar recebe 403, e essa própria tentativa vira um evento (`403_acesso_logs`) no log. O Redis tem healthcheck no compose, e o serviço de logs depende dele.

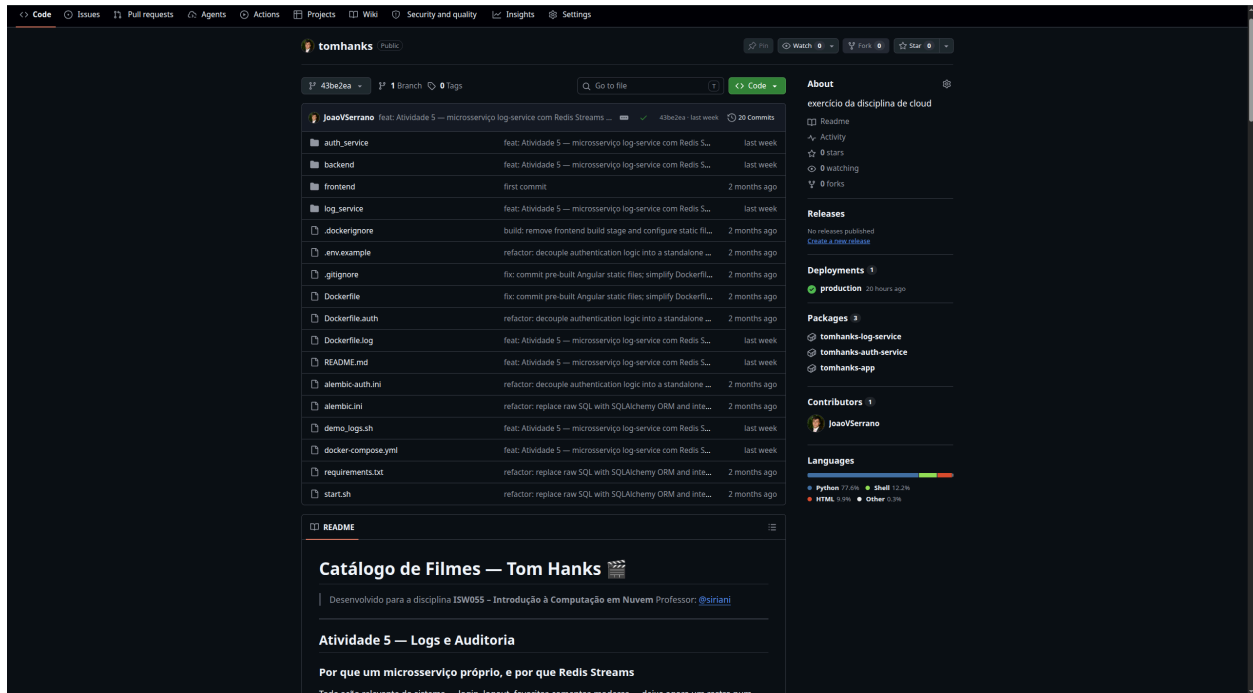


Figura 10: Atividade 5 — evidência da entrega: página do commit 43be2ea no GitHub, com hash, data/hora e nome do repositório visíveis

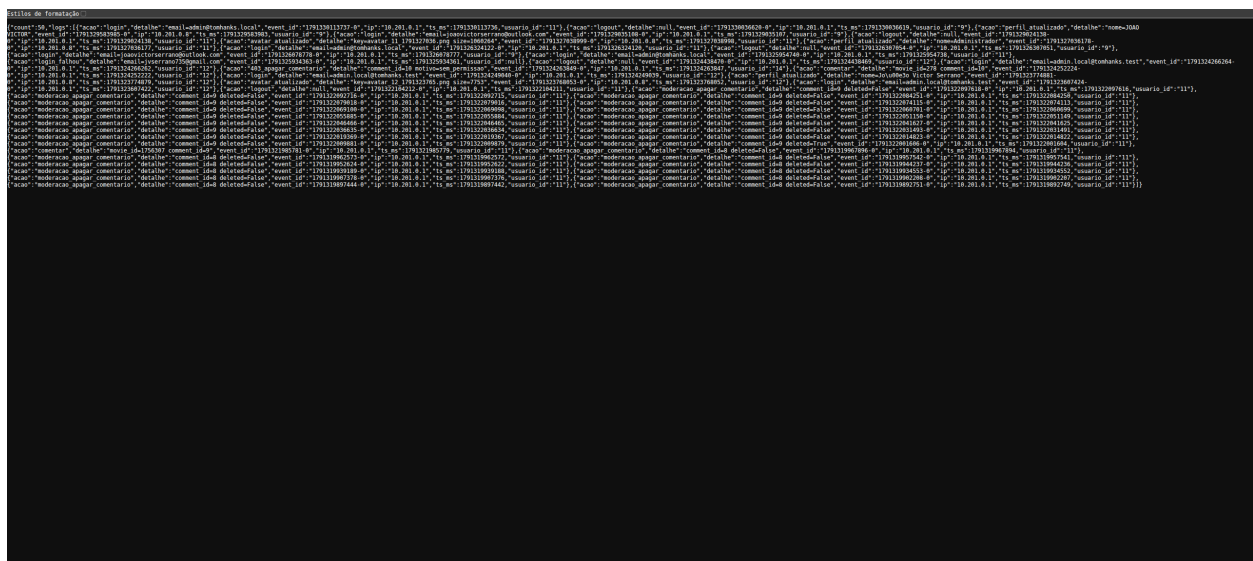


Figura 11: Atividade 5 — resultado: consulta de logs feita pelo administrador mostrando histórico de ações realizadas na plataforma

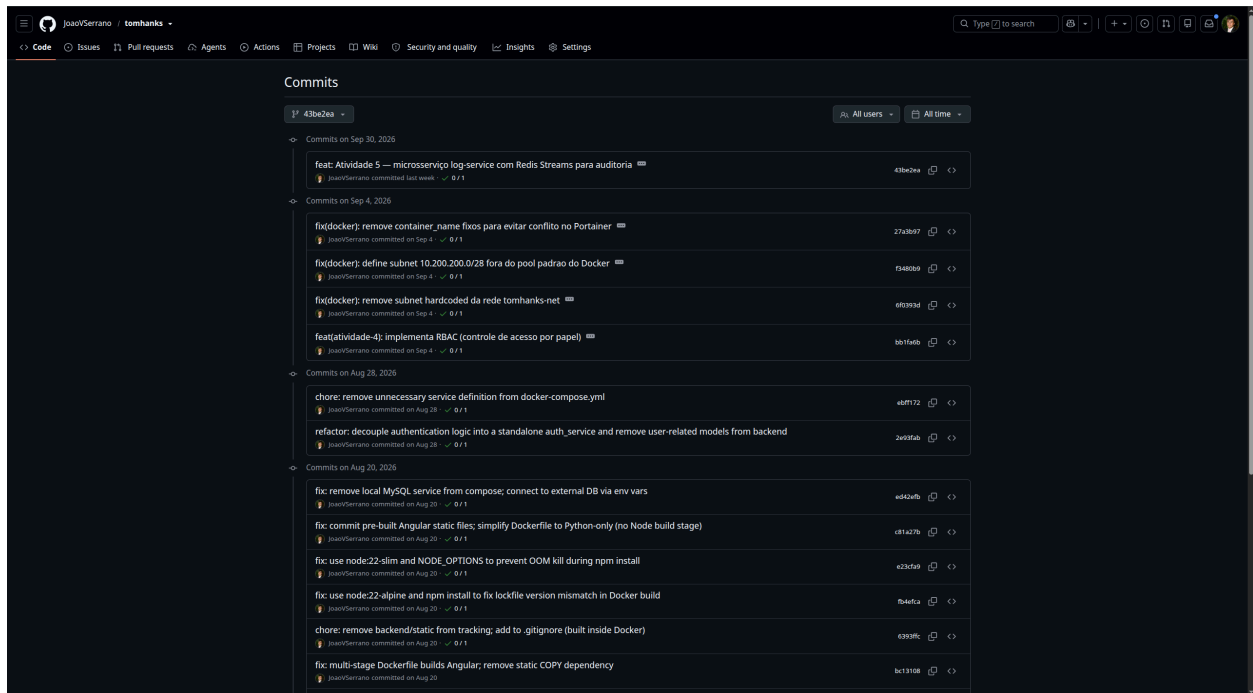


Figura 12: Atividade 5 — detalhe dos commits até a entrega 43be2ea

Dificuldades e como foram resolvidas. A primeira versão

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	30/09/2026 20:04
Situação	entregue
Evidência	GitHub — commit 4c99189 + print do perfil com foto
Link	https://github.com/JoaoVSerrano/tomhanks/commit/4c99189

O que foi feito. O sistema ganhou uma página de perfil em que o usuário edita nome e biografia e envia uma foto. O arquivo da imagem vai para um bucket do MinIO (armazenamento de objetos compatível com S3), e no banco relacional ficam apenas a biografia e a referência da imagem, em duas colunas novas de usuarios: bio e avatar_key. As rotas são GET e PUT /api/profile/<id>, para ler e editar, POST /api/profile/#raw("<id>")/avatar para o upload e GET /api/profile/avatar/#raw("<chave>") para entregar a imagem ao navegador. O envio só aceita arquivos de imagem, não vazios e de até 5 MB.

Quem não é o dono do perfil recebe 403 ao tentar editá-lo ou trocar a foto, e a tentativa é registrada no log de auditoria. O MinIO roda como mais um contêiner do compose, com volume próprio, healthcheck e portas apenas na rede interna, de modo que o navegador nunca fala direto com ele.

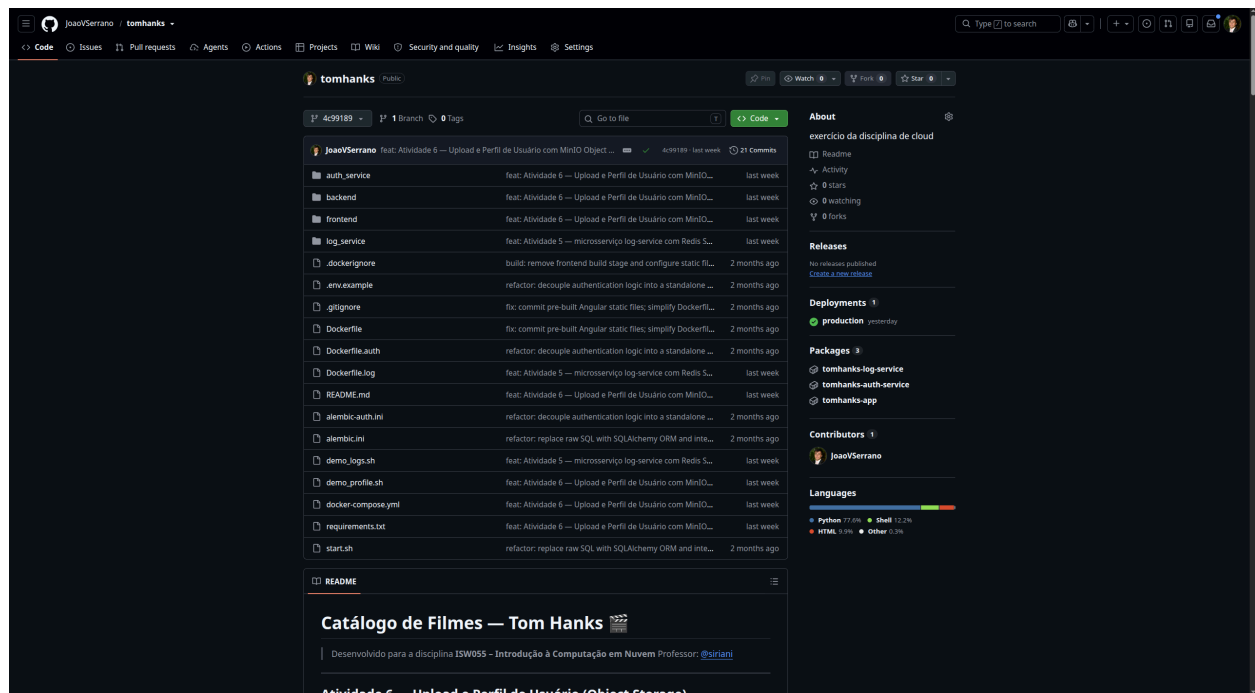


Figura 13: Atividade 6 — evidência da entrega: página do commit 4c99189 no GitHub, com hash, data/hora e nome do repositório visíveis



Figura 14: Atividade 6 — resultado: página de perfil exibindo a foto enviada, com nome e biografia

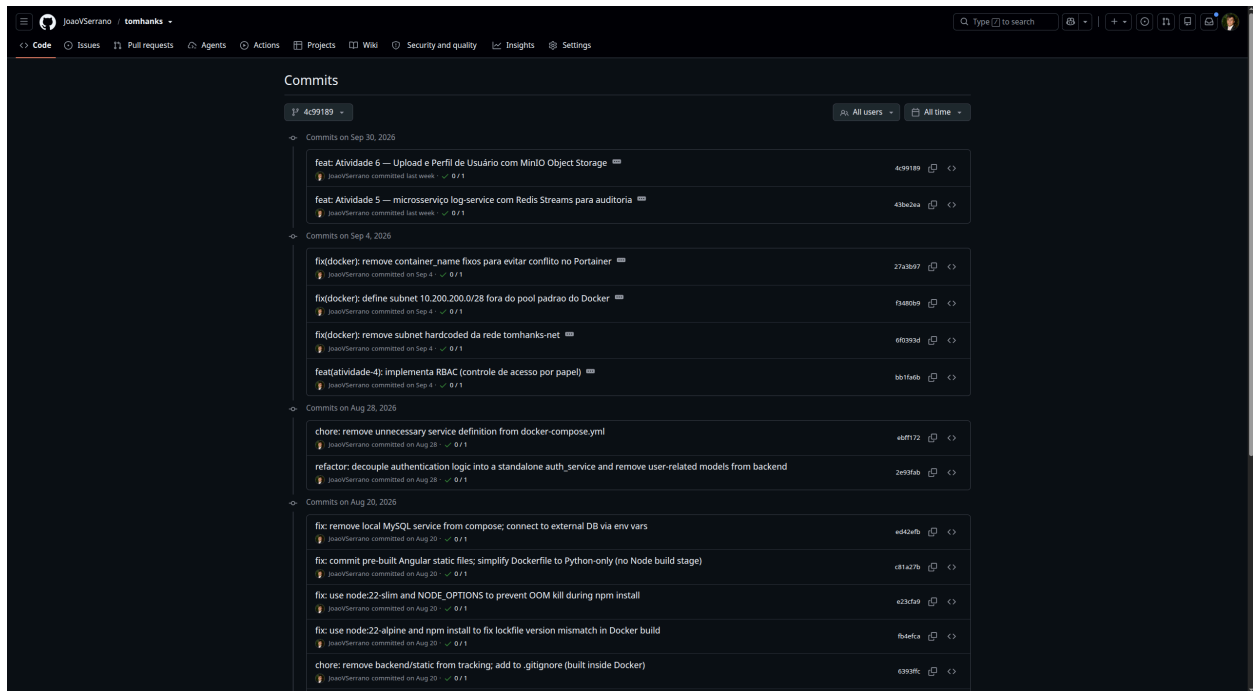


Figura 15: Atividade 6 — detalhe dos commits até a entrega 4c99189

Dificuldades e como foram resolvidas. Os problemas apareceram na implantação no Portainer e foram resolvidos nos dias seguintes. Em 30/09, o commit 1b7b301 (22:19) aprimorou a tela de perfil e corrigiu a persistência do login. Em 02/10, o commit bf9c884 (18:24) atualizou a imagem do MinIO e removeu um conflito de porta com o Portainer, o 85f5efc (18:45) evitou o recarregamento da página no login e passou a enviar as credenciais (cookie) na requisição de upload, e o 3793f45 (18:53) ajustou o usuário do contêiner do MinIO para ele ter permissão de escrita no volume que já existia. Todas essas correções ocorreram antes do prazo de 02/10 às 23:59.

5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.
Data planejada	sem prazo
Data realizada	05/10/2026 21:26
Situação	entregue
Evidência	GitHub — commit c83b3b4, README com menção a github.com/siriani, print do Swagger UI com Try it out executado
Link	https://github.com/JoaoVerrano/tomhanks/commit/c83b3b4

O que foi feito. Cada serviço passou a publicar a sua própria especificação OpenAPI 3.0 e uma Swagger UI (/apidocs): o gateway documenta 23 operações do catálogo, da autenticação, do perfil e da administração, o auth-service documenta 14 e o log-service documenta 3, incluindo o esquema de segurança do token interno. Como o auth-service e o log-service não têm porta publicada, o gateway reúne as três definições em uma única Swagger UI com seletor, buscando as especificações internas com tempo limite curto e mostrando uma mensagem amigável se algum serviço estiver fora do ar. As especificações ficam

em arquivos `swagger_spec.py`, e os arquivos `openapi.json` e `openapi.yaml` da raiz são gerados por um script (`scripts/generate_openapi.py`). Testes automatizados comparam as rotas documentadas com as rotas reais de cada serviço, para a documentação não se afastar do código.

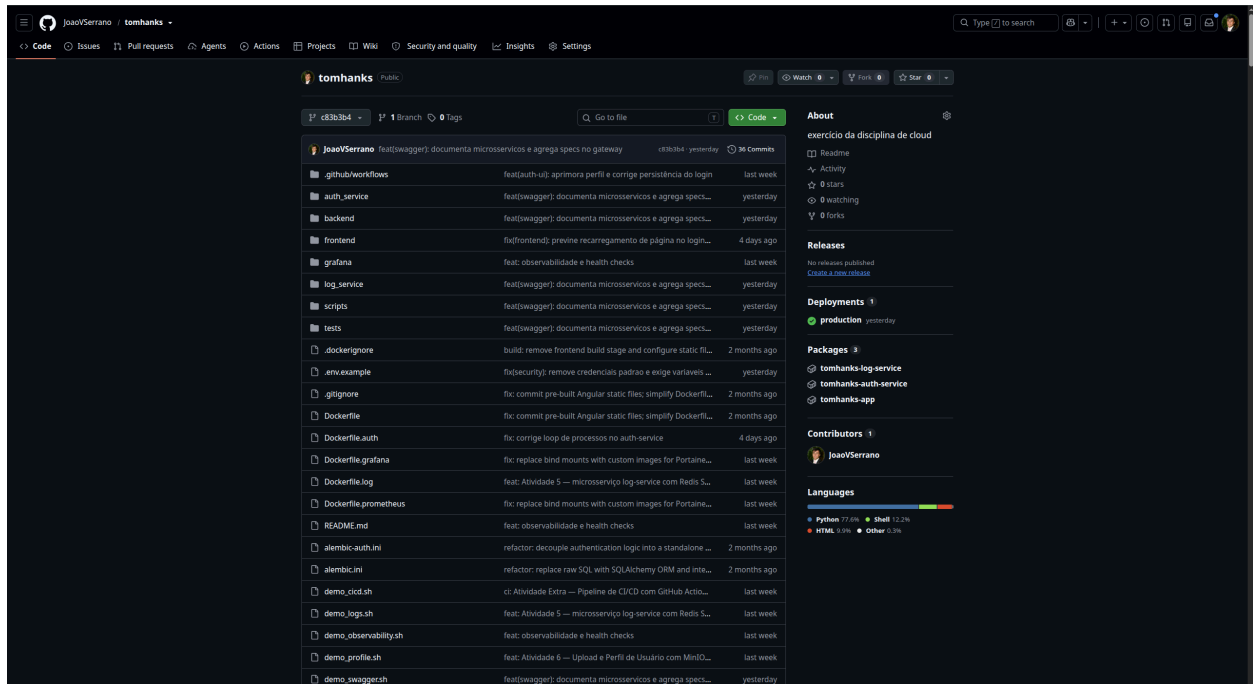


Figura 16: Atividade E1 — evidência da entrega: página do commit c83b3b4 no GitHub, com hash, data/hora e nome do repositório visíveis

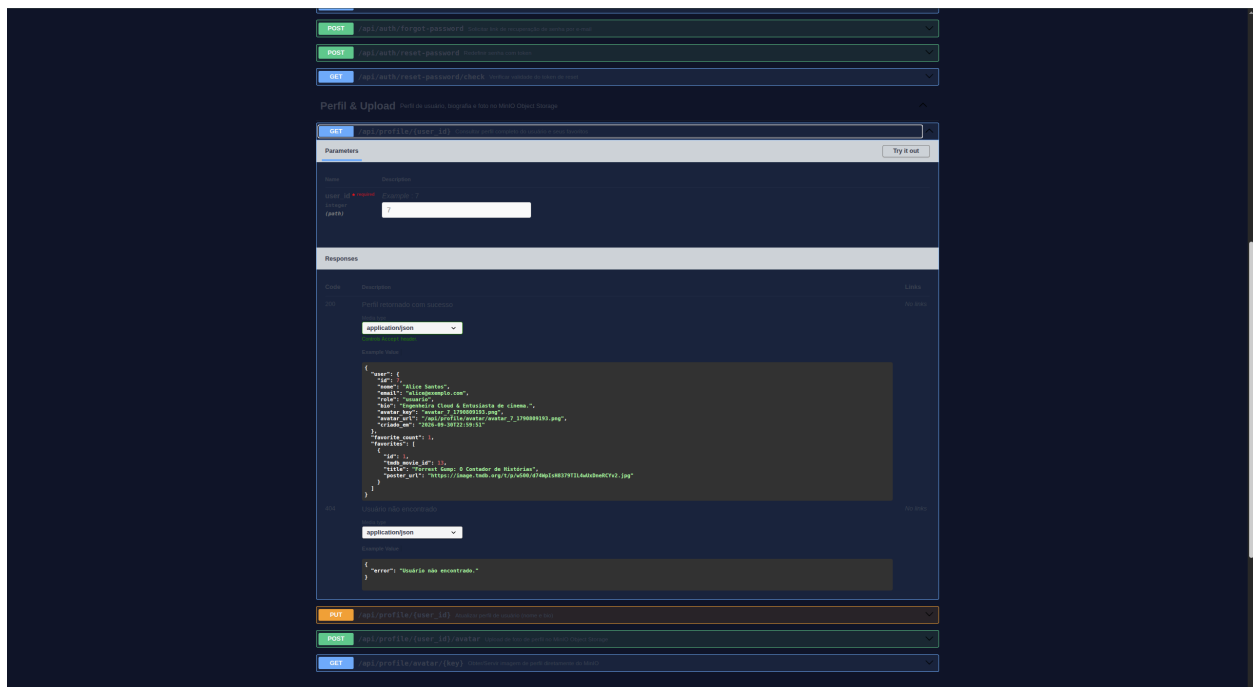


Figura 17: Atividade E1 — resultado: Swagger UI com o seletor de serviços e um “Try it out” executado com resposta

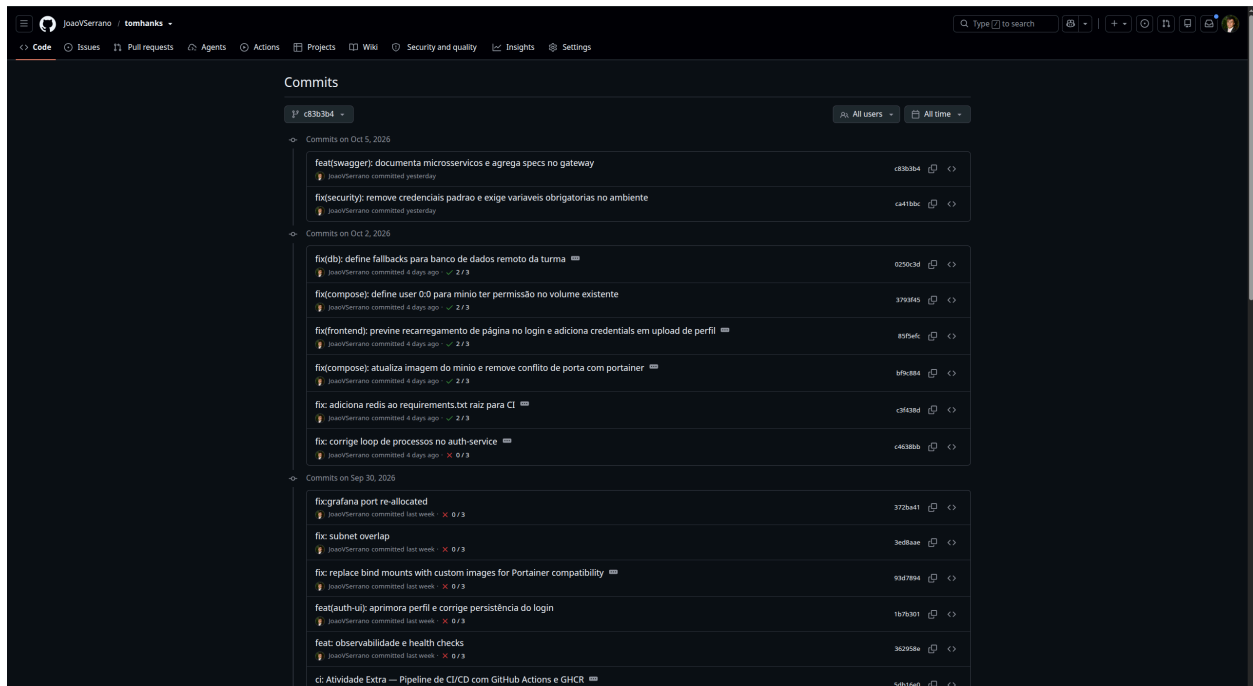


Figura 18: Atividade E1 — detalhe dos commits até a entrega c83b3b4

Dificuldades e como foram resolvidas. A primeira versão (044b15b, 30/09 20:13) concentrava toda a documentação em uma única especificação no gateway. Ela descrevia as rotas dos outros serviços, mas nenhum deles publicava documentação própria. Por isso a data de entrega aqui é a do commit c83b3b4, em que o requisito passou a ser atendido de fato.

5.2 Atividade E2 — CI/CD com GitHub Actions

Descrição	Pipeline que testa e faz deploy a cada push.
Data planejada	sem prazo
Data realizada	05/10/2026 21:27
Situação	entregue
Evidência	GitHub — commit 9d6a20c, README com menção a github.com/siriani, print da execução do workflow na aba Actions
Link	https://github.com/JoaoVSerrano/tomhanks/commit/9d6a20c

O que foi feito. O workflow `.github/workflows/ci-cd.yml` roda a cada push e tem três etapas encadeadas. A primeira (test) instala as dependências e executa a suíte pytest com variáveis fictícias de ambiente. A segunda (build-and-push), só em push na main, constrói as imagens do gateway, do auth-service e do log-service e as publica no GitHub Container Registry. A terceira (deploy) usa o ambiente production do GitHub, dispara por webhook a atualização da stack no Portainer, com tentativas repetidas, e depois consulta o endpoint de saúde do gateway por alguns minutos, falhando se ele não ficar saudável. A URL do webhook fica em um secret do repositório, e se ele não estiver configurado o deploy é ignorado com um aviso, em vez de falhar em silêncio. O `docker-compose.yml` da stack continua construindo as imagens a partir do código (build:).

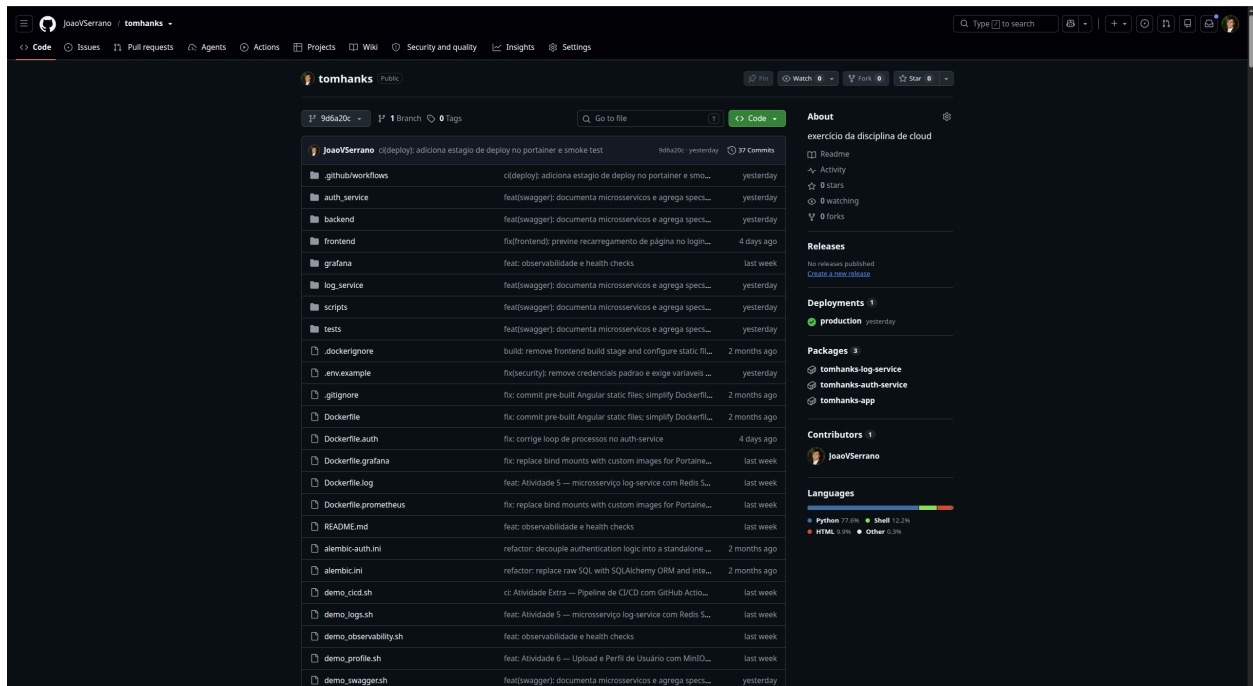


Figura 19: Atividade E2 — evidência da entrega: página do commit 9d6a20c no GitHub, com hash, data/hora e nome do repositório visíveis

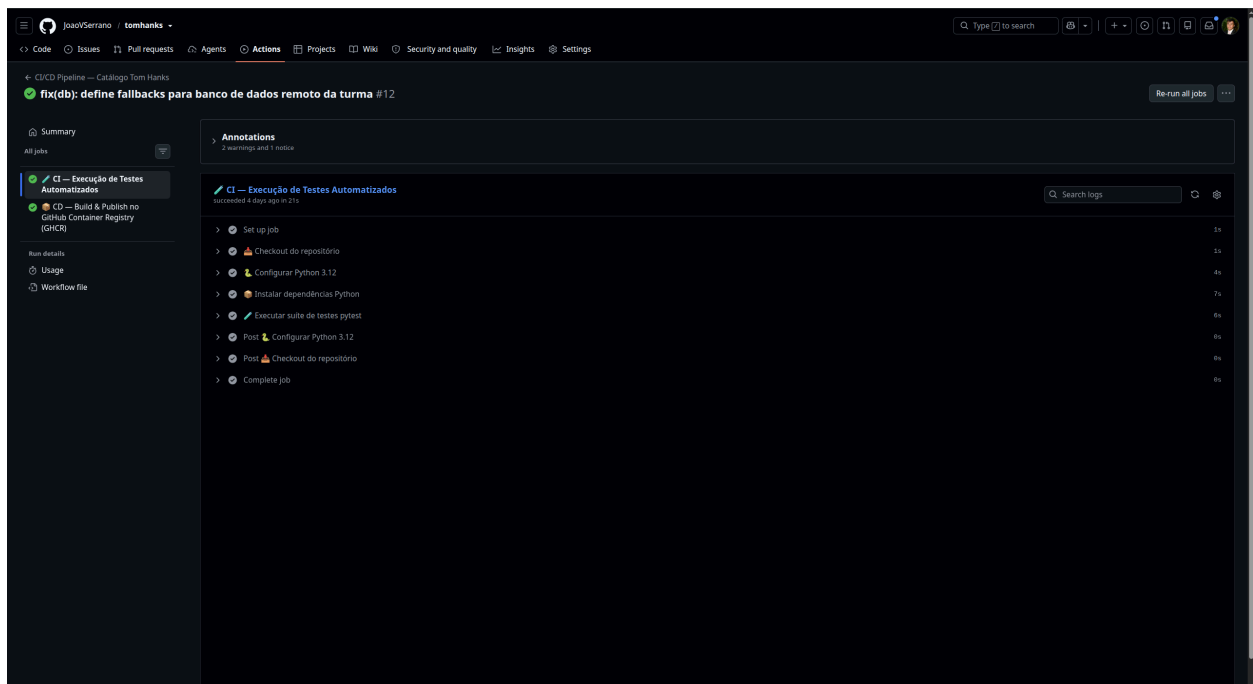


Figura 20: Atividade E2 — resultado: execução do workflow na aba Actions do GitHub com as etapas test, build-and-push e deploy concluídas

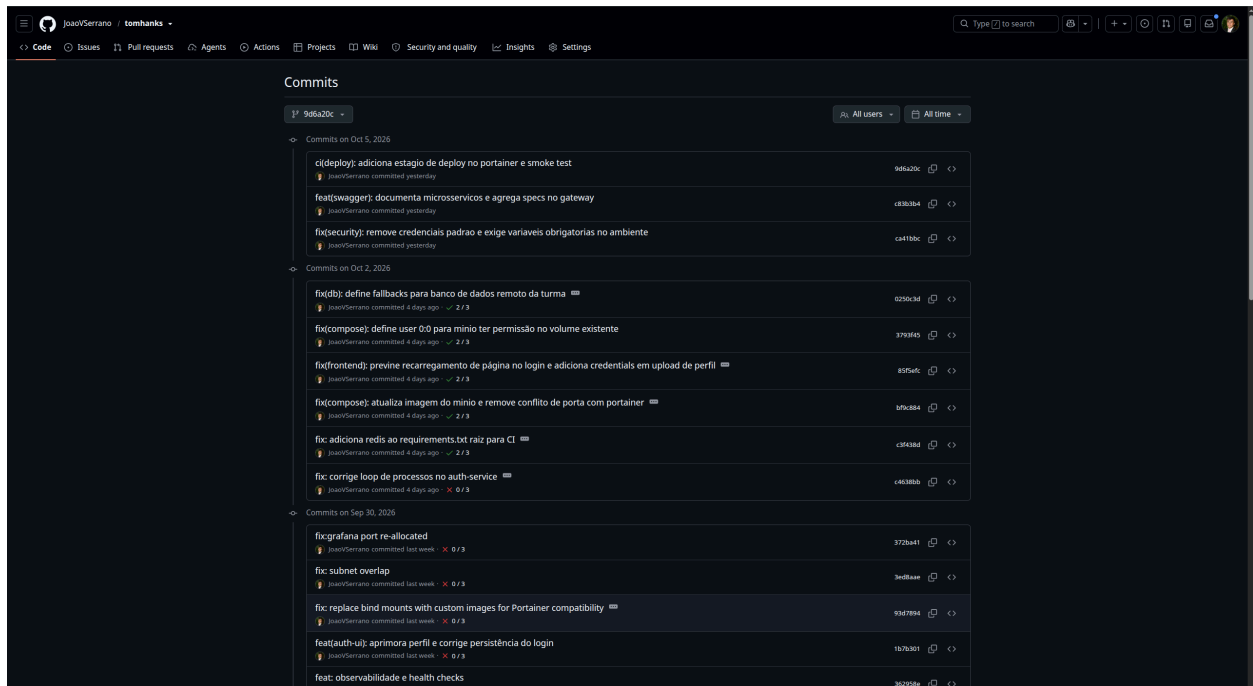


Figura 21: Atividade E2 — detalhe dos commits até a entrega 9d6a20c

Dificuldades e como foram resolvidas. A versão inicial (5db16e0, 30/09 20:17) testava e publicava imagens, mas não fazia deploy. Em 02/10, o commit c3f438d (18:11) acrescentou o redis ao requirements.txt da raiz, dependência que os testes precisavam no CI. Como a etapa de deploy só existe a partir do commit 9d6a20c, esta é a data de entrega.

5.3 Atividade E3 — Observabilidade — health checks e métricas

Descrição	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.
Data planejada	sem prazo
Data realizada	30/09/2026 20:43
Situação	entregue
Evidência	GitHub — commit 362958e, README com menção a github.com/siriani, print do Grafana e do teste de queda do Redis
Link	https://github.com/JoaoVSerrano/tomhanks/commit/362958e

O que foi feito. Cada serviço ganhou um endpoint /health que verifica as suas dependências reais, e o do log-service responde 503 quando o Redis não responde. Os três serviços também expõem /metrics no formato Prometheus (com a biblioteca de métricas para Flask). Um Prometheus coleta essas métricas conforme o prometheus.yml, e um Grafana já sobe com um painel provisionado (grafana/dashboards/observability.json). No compose, healthchecks do Docker acompanham o Redis, o log-service, o MinIO e os demais serviços.

O script demo_observability.sh demonstra a reação à queda: ele para o Redis, mostra o log-service respondendo 503 e o docker compose ps marcando o contêiner como unhealthy, e depois religa o Redis e mostra o serviço voltando a healthy. Vale notar que o contêiner é marcado como não saudável, mas o Docker não o reinicia sozinho por causa disso.

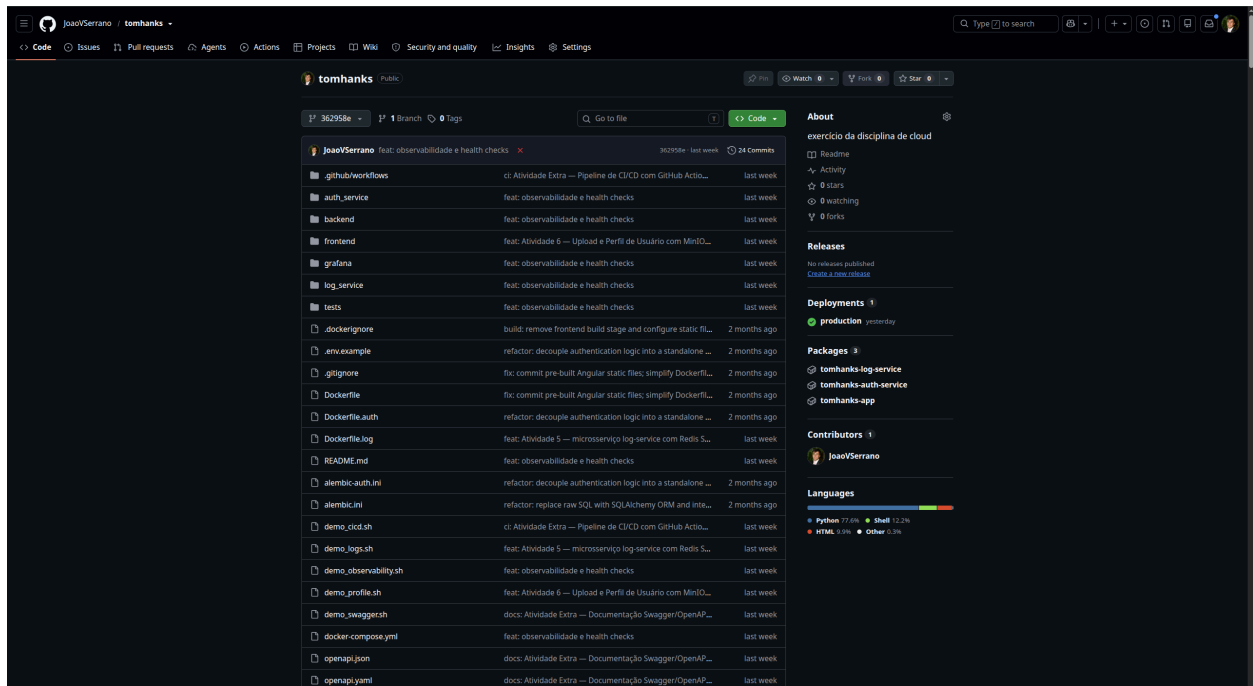


Figura 22: Atividade E3 — evidência da entrega: página do commit 362958e no GitHub, com hash, data/hora e nome do repositório visíveis



Figura 23: Atividade E3 — resultado: painel do Grafana com métricas e a saída do `demo_observability.sh` mostrando o `log-service` em 503/unhealthy com o Redis parado

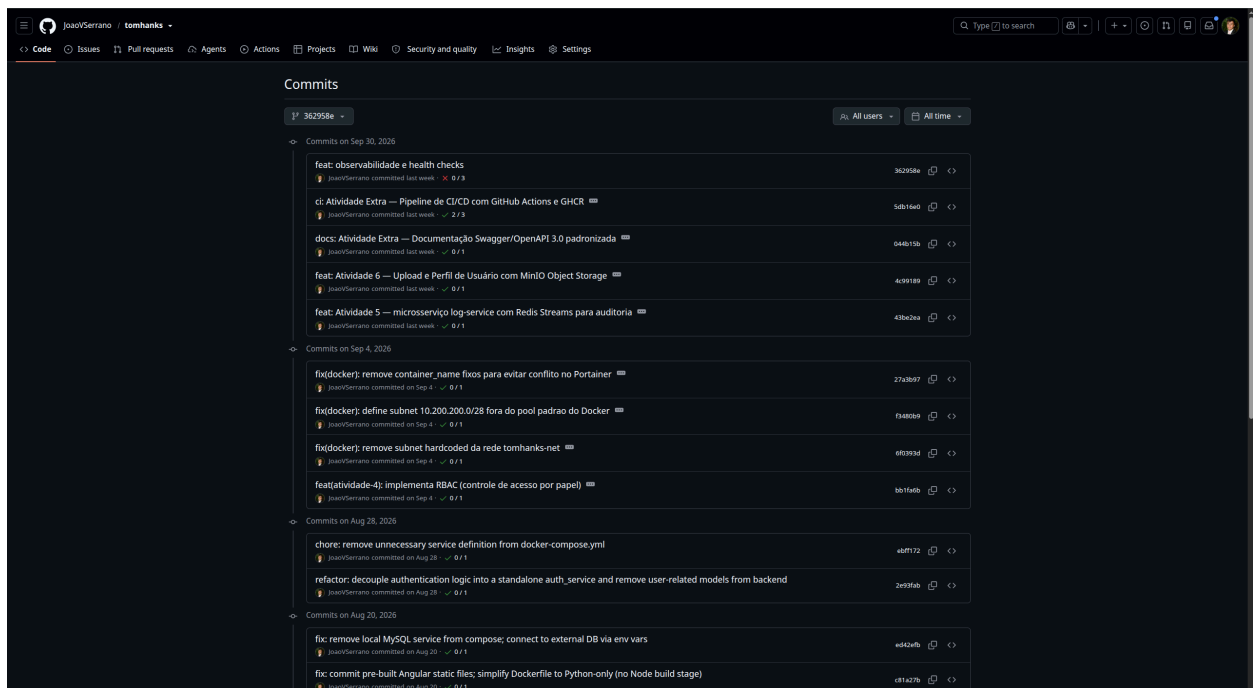


Figura 24: Atividade E3 — detalhe dos commits até a entrega 362958e

Dificuldades e como foram resolvidas. Na implantação no Portainer houve três correções em 30/09: a troca dos arquivos de configuração montados do host por imagens próprias de Prometheus e Grafana, por compatibilidade com o Portainer (93d7894, 22:25), a correção de um conflito entre sub-redes (3ed8aae, 22:27) e a mudança da porta do Grafana (372ba41, 22:30). Em 02/10, o commit c4638bb ajustou os parâmetros de healthcheck do auth-service (intervalo e período de inicialização) e tirou o MinIO do critério de saúde, deixando só o banco como dependência crítica.

6 Considerações finais

O bimestre mostrou na prática como uma aplicação cresce quando é dividida em serviços: separar autenticação, logs e armazenamento de arquivos deixou cada parte mais simples, mas exigiu pensar em rede, em contratos entre serviços e em segurança de comunicação interna. O mais difícil foi a distância entre rodar localmente e rodar no Portainer, onde sub-redes, nomes de contêiner, portas e permissões de volume causaram a maior parte das correções.

O histórico também mostra um padrão a corrigir: várias entregas foram fechadas muito perto do prazo, como a Atividade 2, às 23:59, e a Atividade 5 acabou atrasada. Para o próximo bimestre, a intenção é fazer commits menores e mais frequentes, subir cada atividade no ambiente de implantação logo no início, e não por último, e manter segredos fora do código desde o primeiro commit.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

João Victor Serrano

Pompeia, 07/10/2026